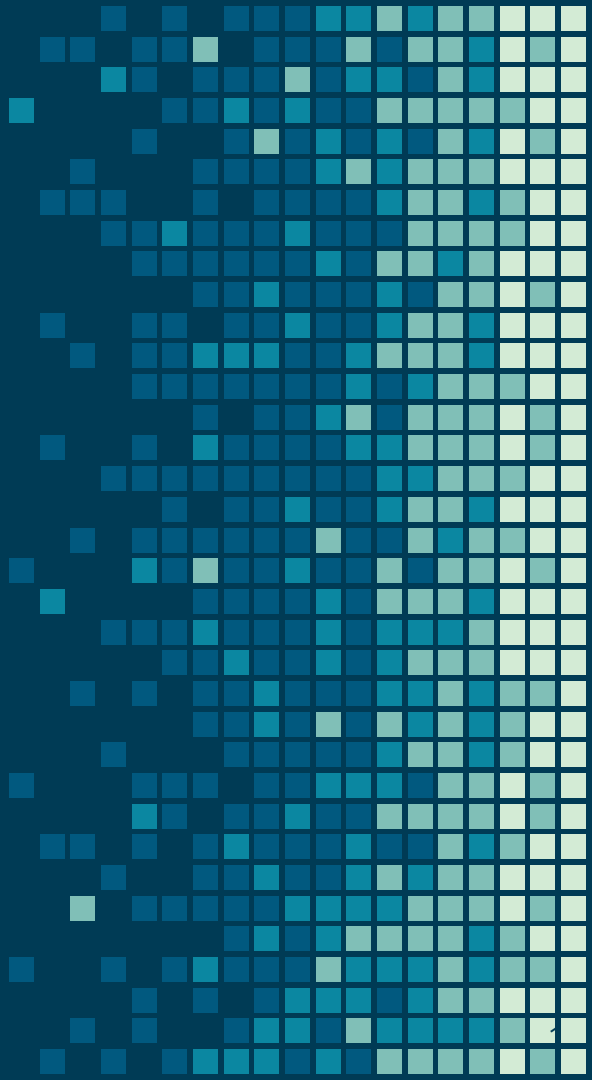
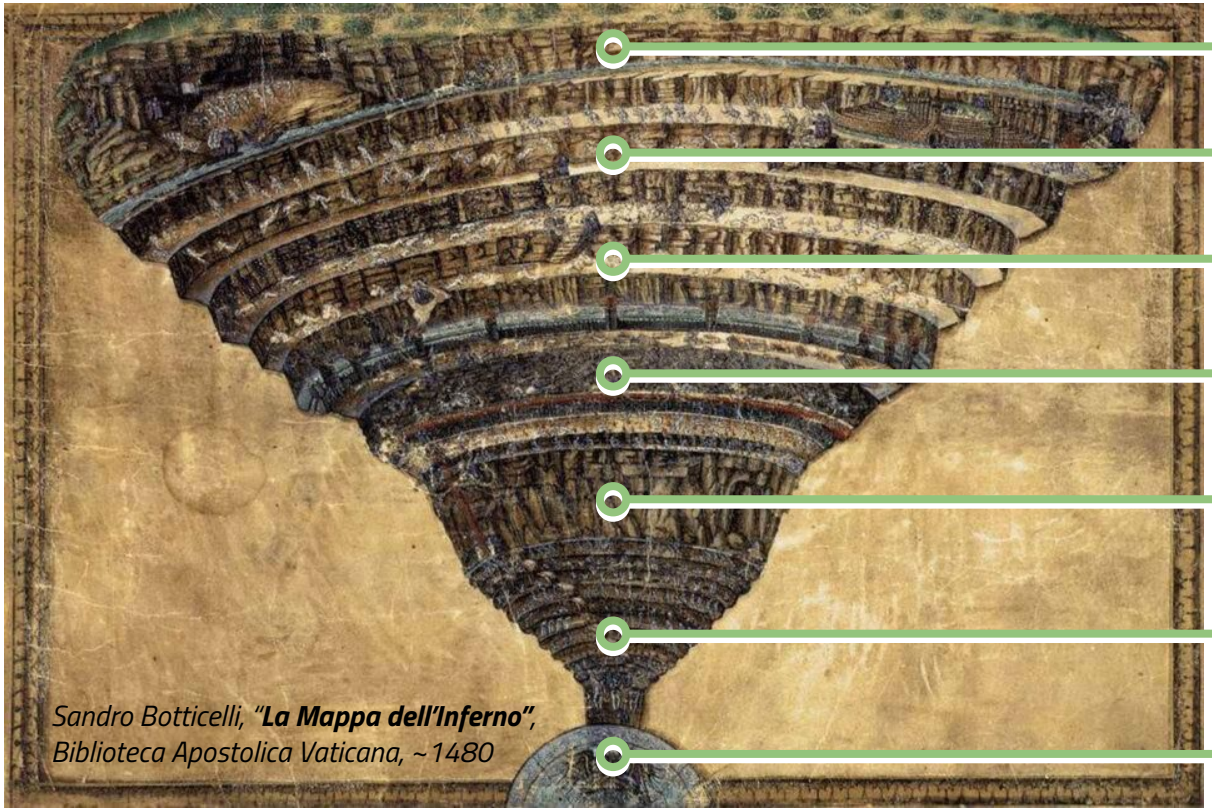
EMBEDDED CONTROL

Dr. Luca De Cicco
luca.decicco@poliba.it
https://c3lab.poliba.it/LDC



Introducing STM32CubeIDE/MX and HAL

Where are we?



*Sandro Botticelli, "**La Mappa dell'Inferno**", Biblioteca Apostolica Vaticana, ~1480*

RTOS

Programming with HAL

Programming with CMSIS

Programming registers

Peripherals Hw

MCU Hw

ARM Cortex-M

PATH TO ~~REDEMPTION~~ ENLIGHTENMENT

Two Ways to program the MCU

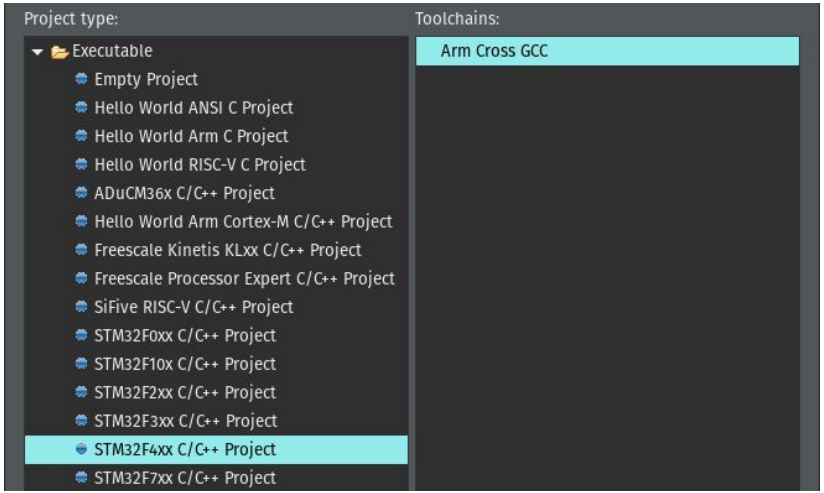
- The hard way: programming at the register level using helper macros and defines (CMSIS)
  - Advantages: the code is as close as possible to the HW, if well coded best possible performances and lowest latencies
  - Drawbacks: code is difficult to maintain, difficult to port to different MCUs even of the same vendor
- The not-so-hard way: programming using an Hardware Abstraction Layer (HAL)
  - Advantage: code is easier to understand and maintain. Very high portability across different families of the same vendor
  - Drawback: slower code, bugs could be due to the implementation of the HAL

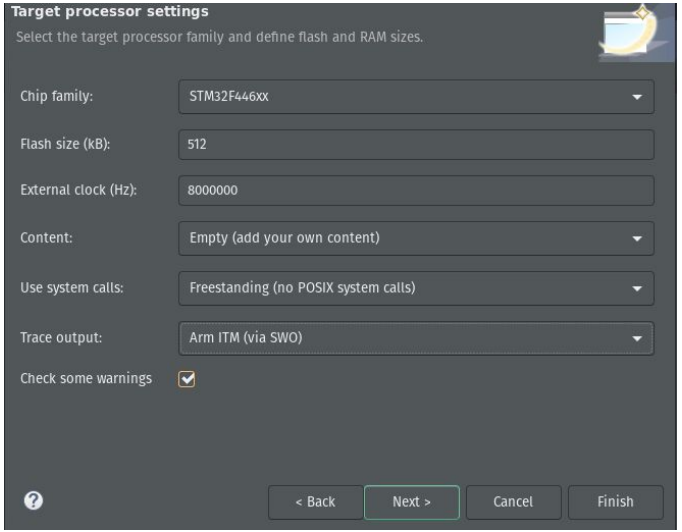
STM32CubeIDE+STM32CubeMX programming

- To program at the register-level:
  - It is not needed any other tool.
  - This procedure do not create any code for you, it just initializes the project with some useful header files, the main source file, and a linker script
- To program using HAL the CubeMX tool has to be used.
- CubeMX allows to configure:
  - PINs and assign them to AFs or define them as GPIO I/O
  - Clock distribution to the CPU core and the peripherals
  - The tool generates code that automatically initializes all your peripherals and the clocks

Generate Empty Project in STM32CubeIDE

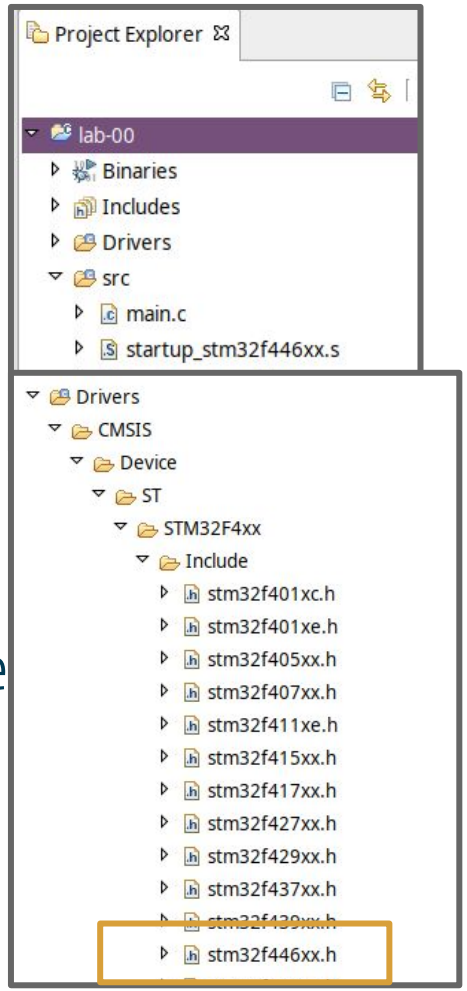
- Open STM32CubeIDE and click on **File→New→C Project**
- Select **C project**, give a name to the project (for instance **lab-00**) and click on **Next**
- Now select the target (i.e. the STM32F4XX) and click **Next**





Project Structure

- The project explorer shows the project structure
- src/ directory contains the source files (`*.c`, `*.s` - asm)
- Drivers/ contains header files that are compliant to **CMSIS** (see next slides) that defines a "standardized" view of the MCU peripherals and core
- In our case the file that we need to look is **`stm32f446xx.h`**



stm32f446xx.h (1/2)

Huge header file (~8k LOC) containing:

- **data types** mapping 1:1 peripheral registers using the naming convention `PERIPH_TypeDef` (f.i., `ADC_TypeDef`, `GPIO_TypeDef`,...)
- **Mapping addresses** of all peripherals and registers, f.i.:
  `#define PERIPH_BASE ((uint32_t)0x40000000)`
- **Peripheral declarations** casting mapping addresses to peripheral registers data structures, f.i.,:

```
#define CAN1  ((CAN_TypeDef *) CAN1_BASE)
#define RCC   ((RCC_TypeDef *) RCC_BASE)
```

stm32f446xx.h (2/2)

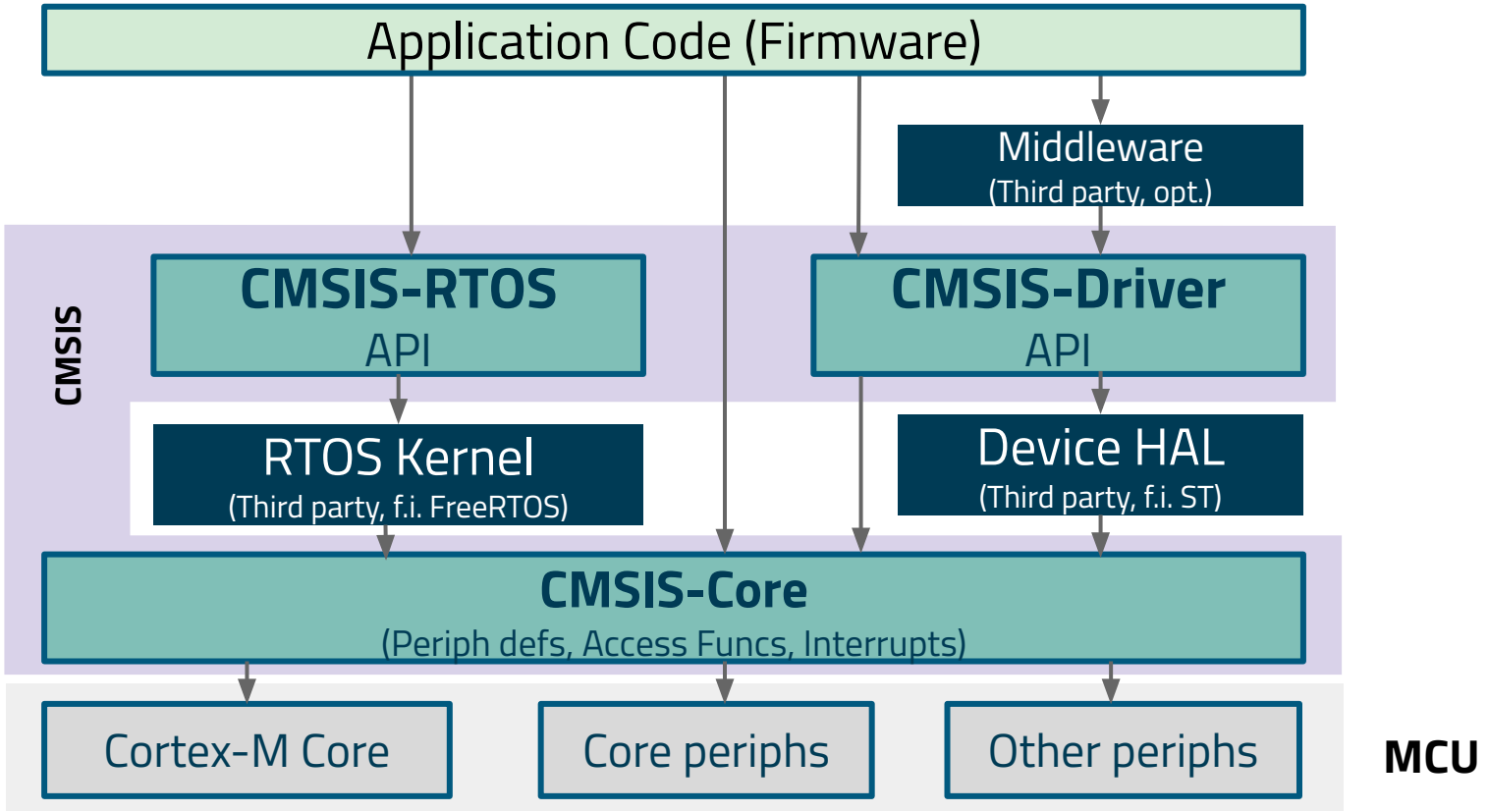
- A huge list of bitmask definitions to access single bits in a register (starting at ~1200 line ending at ~8150 line) and grouped by peripheral (~300 defines for ADC)
- A list of Macros implementing boolean functions useful for error checking
- **IMPORTANT**: at line ~188 you have the include:
  `#include "core_cm4.h" /* Cortex-M4 proc */`
- This header file is common to **ANY MCU in the market** (not only by ST) **using an ARM Cortex-M4** core and compliant to the CMSIS standard

CMSIS

So what is **CMSIS**?

- CMSIS stands for **Cortex Microcontroller Software Interface Standard**
- Vendor-independent hardware abstraction layer for ARM Cortex-M processors
- Enables consistent device support and simple software interfaces to processor and peripherals
- Allows reusability, reducing the learning curve for microcontroller developers (don't have to start from scratch each time you change vendor!)

CMSIS Compliant Software



CMSIS Components

- **CMSIS-CORE**: API for the Cortex-M processor core and peripherals. It provides a standardized interface for Cortex-M0/3/4/7
- **CMSIS-Driver**: defines generic peripheral driver interfaces. The API connects MCU periphs to higher level middleware which might implement communication stack (f.i., TCP/IP), filesystems, etc
- **CMSIS-RTOS**: Common API for Real-Time Operating Systems, providing a standardized programming interface which is portable to many RTOS

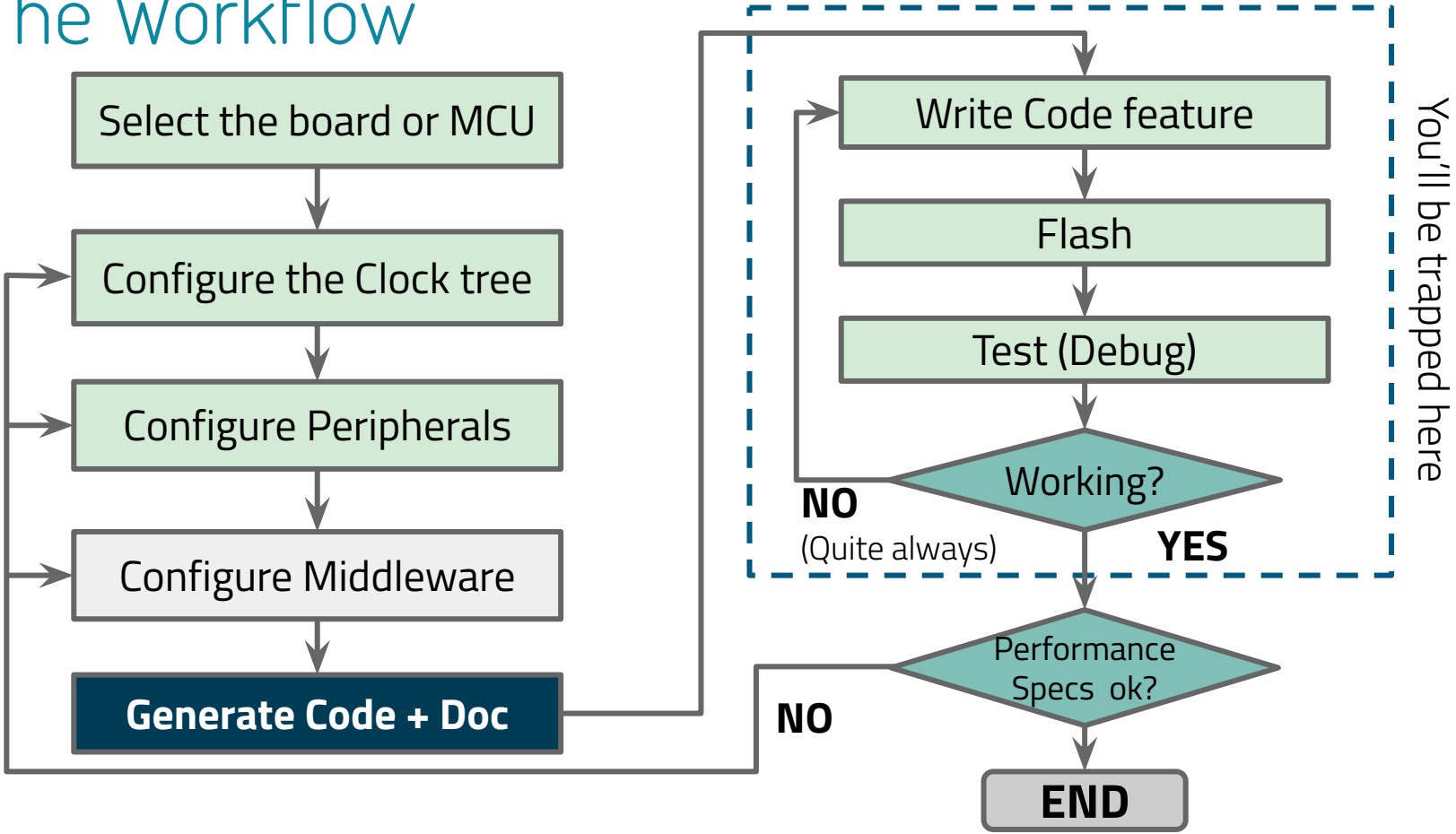
Programming with HAL

- The **Hardware Abstraction Layer** abstracts the MCU and provides a set of APIs to access all its peripherals
- It provides a convenient way to program the MCU without having to directly work at the registers level
- Since the HAL has to support all the MCUs of the STM32 family, a tool is needed to tailor the library for a particular MCU
- The CubeMX is a free tool (now integrated in the IDE) that can be used to configure the MCU (pins, clock, etc) and generate an empty project skeleton

CubeMX features

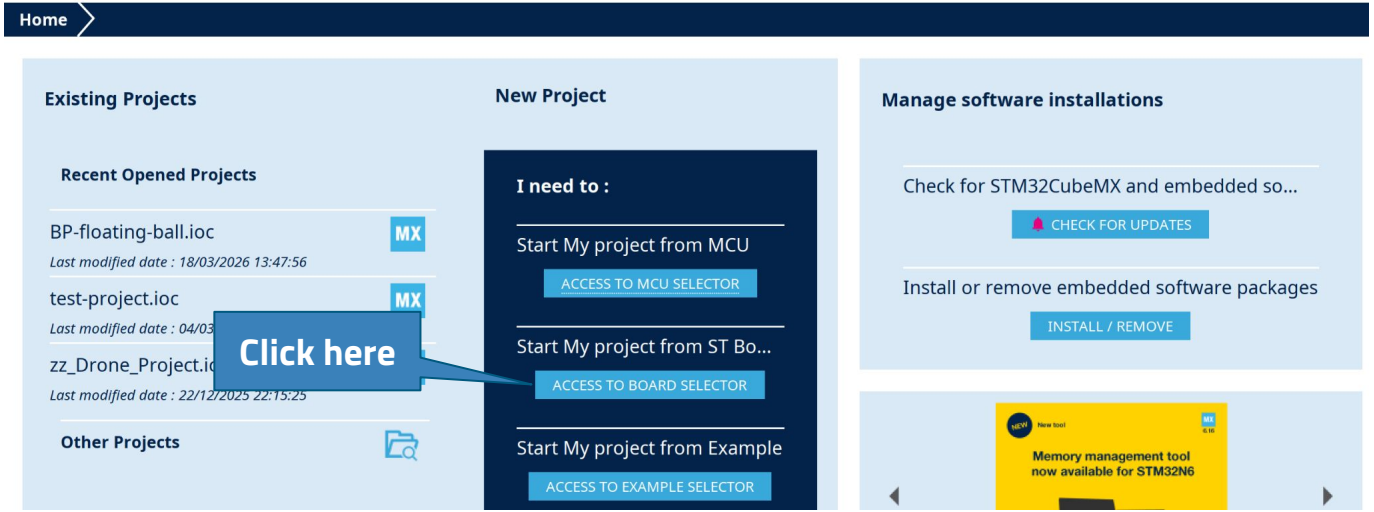
- Configuration of the MCU pins, clock tree and operating modes for the selected peripherals and middleware
- Generation of pin configuration report (a PDF) for board designers. **Very useful for documentation!**
- Generation of a complete project with all the necessary libraries and initialization C code
- The generated project can be directly open in the STM32CubeIDE (other IDEs are also supported)

The Workflow



Creating a Project with STM32CubeMX

- Open the STM32CubeMX



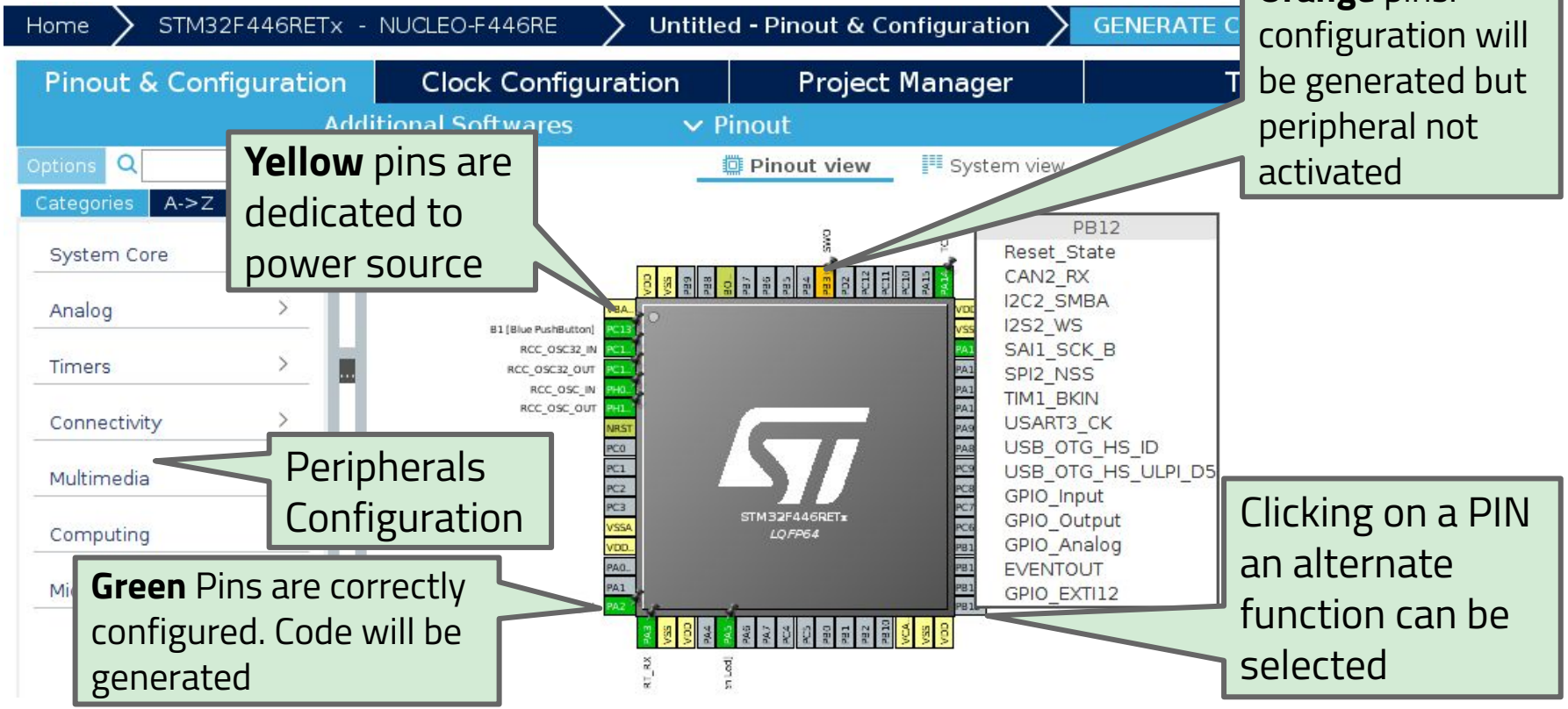
Example: select Nucleo-F446RE board



Example: select Nucleo-F446RE board

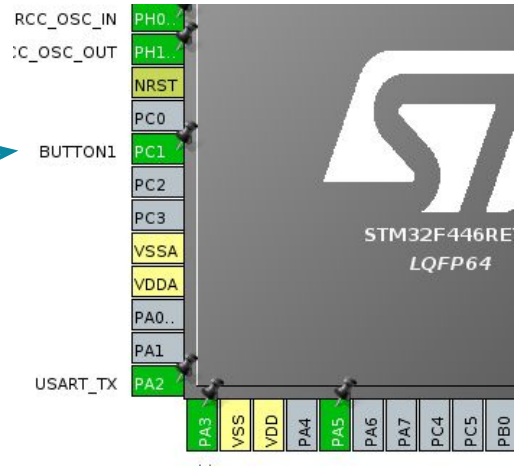


The CubeMX GUI: Pinout & Config



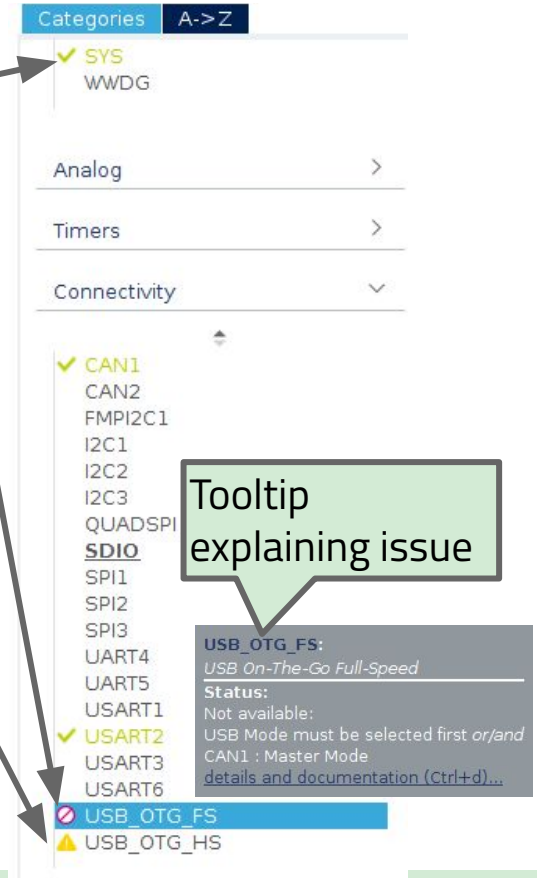
Configuring **PC1** as GPIO Digital Input

- Click on **PC1** pin and then click on **GPIO_Input**
- A mnemonic name of the pin can be given by right-clicking on the PIN and clicking on “Enter User Label” (for instance **BUTTON1**)
- Click on “System Core” (left panel) an then select “GPIO”
- A table with all the PINs configured as GPIOs (input or output) will appear
- Click on the PIN you want to configure (PC1)

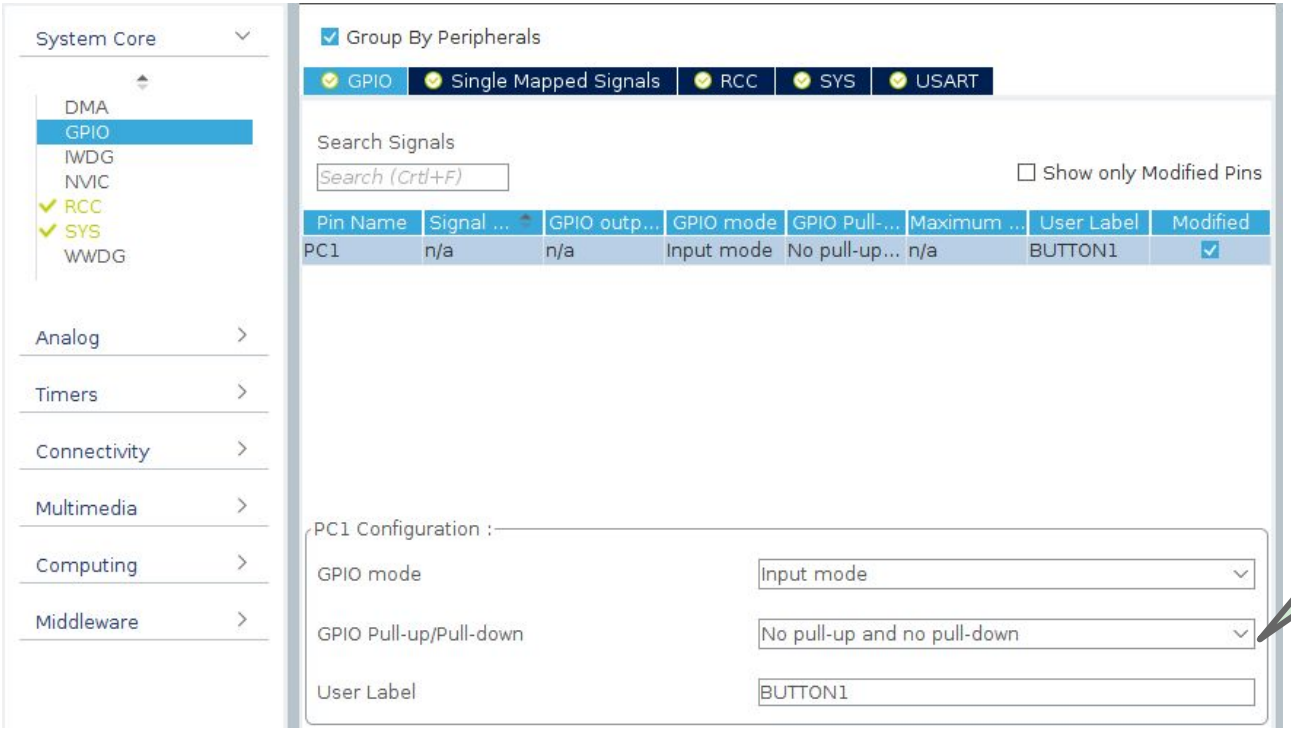


Understanding the Peripherals Panel

- Green *check* symbol ☐ means that the peripherals is configured and all other modes are available
- A *stop* symbol ⊘ means that the peripheral cannot be configured due to conflict with other peripherals
- A *warning* symbol ⚠ means that one of the modes of the peripheral are not available due to conflict with other peripherals.
  - If the name of periph is **black** it is not used
  - if **green** it is correctly configured
- In case of conflicts by hovering with the mouse over the peripheral a tooltip is shown explaining the issue (see figure)



Configuring **PC1** as GPIO Digital Input

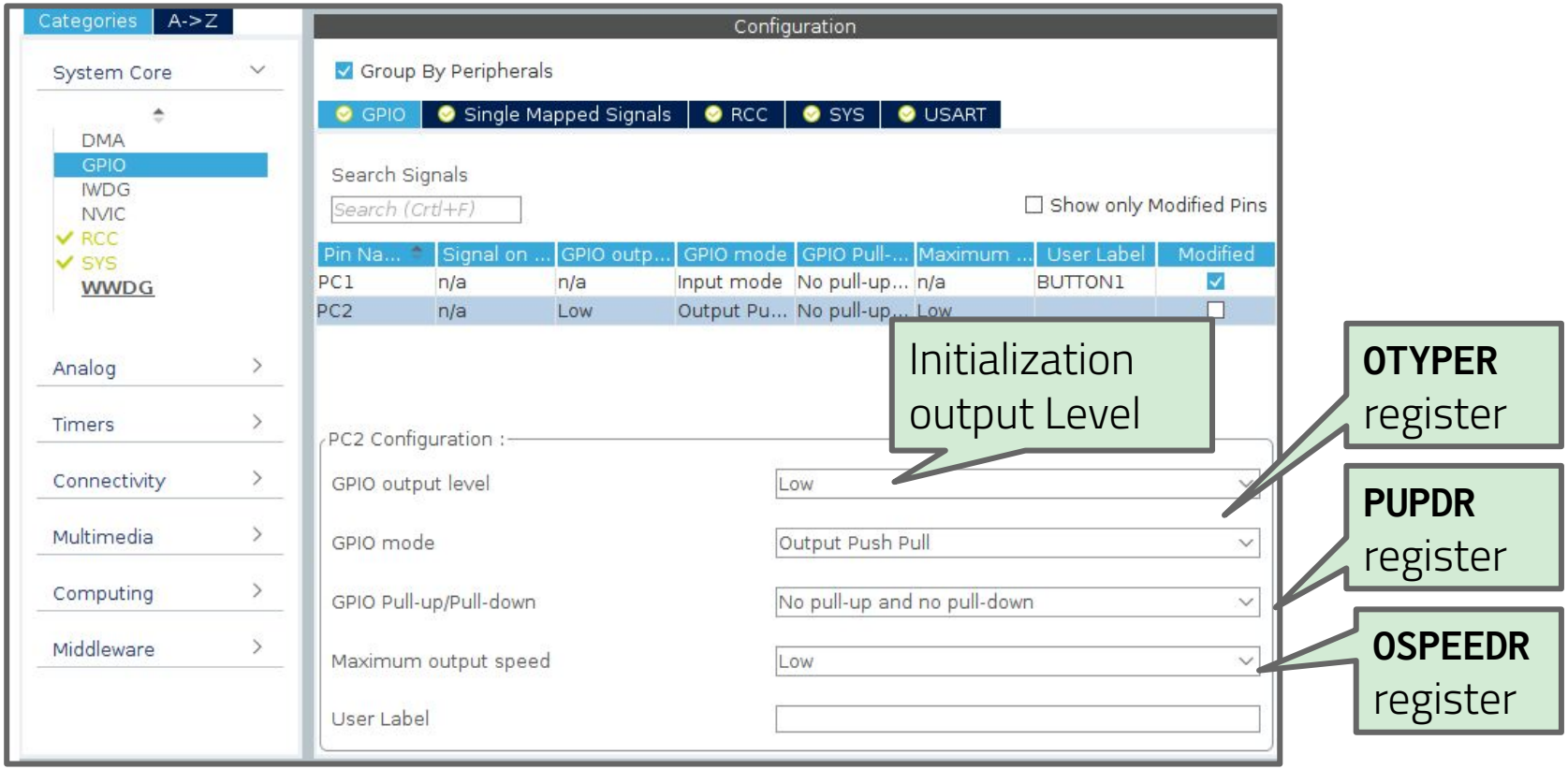


This sets the bit 1 of **PUPDR** register of GPIO port C

Configuring **PC2** as a GPIO Digital Output

- Similarly to the configuration of PC1, click on the pin and select GPIO_Output, then right click on the pin and give it a name using “Enter Label” (f.i., EXT_LED)
- Now you can configure the digital output using the same procedure used for the GPIO Digital Input
- Now, you will be able to also configure registers that are specific to digital output pins

Configuring **PC2** as GPIO Digital Output



Project Manager

The Project Manager tab allows configuring:

- Name and location of the project on the disk
- The way the project skeleton is created/re-generated
- Advanced generation settings:
  - Allows to select at a per-peripheral level whether to generate HAL code or Low Level code (still higher than registers!)

Suggested project settings

Project tab:

- Set project name and location
- IMPORTANT - toolchain/IDE: **set to STM32CubeIDE**
- Leave all other settings to default

Code generator:

- STM32Cube firmware library package: select “Copy only the necessary library files”
- Generated Files: also select “Generate peripheral initialization as a pair of ‘.c/.h’ files per peripheral” (optional but recommended)

Advanced settings: do not touch anything

Code generation

When clicking “Generate code” the tool generates the project skeleton:

- peripheral drivers specific to the value line of your MCU (f.i., STM32F4xxx) are copied in the project tree
- it generates initialization code to set the peripherals as configured through the CubeMX tool

HUGE Warning: the CubeMX tool may overwrite changes you have made to the sources!

Avoid getting user code being overwritten

- Generated code includes "**USER CODE**" sections
- Those sections are **protected**, i.e., the tool cannot touch them (cannot overwrite when re-generating code)
- These sections are delimited by two comments

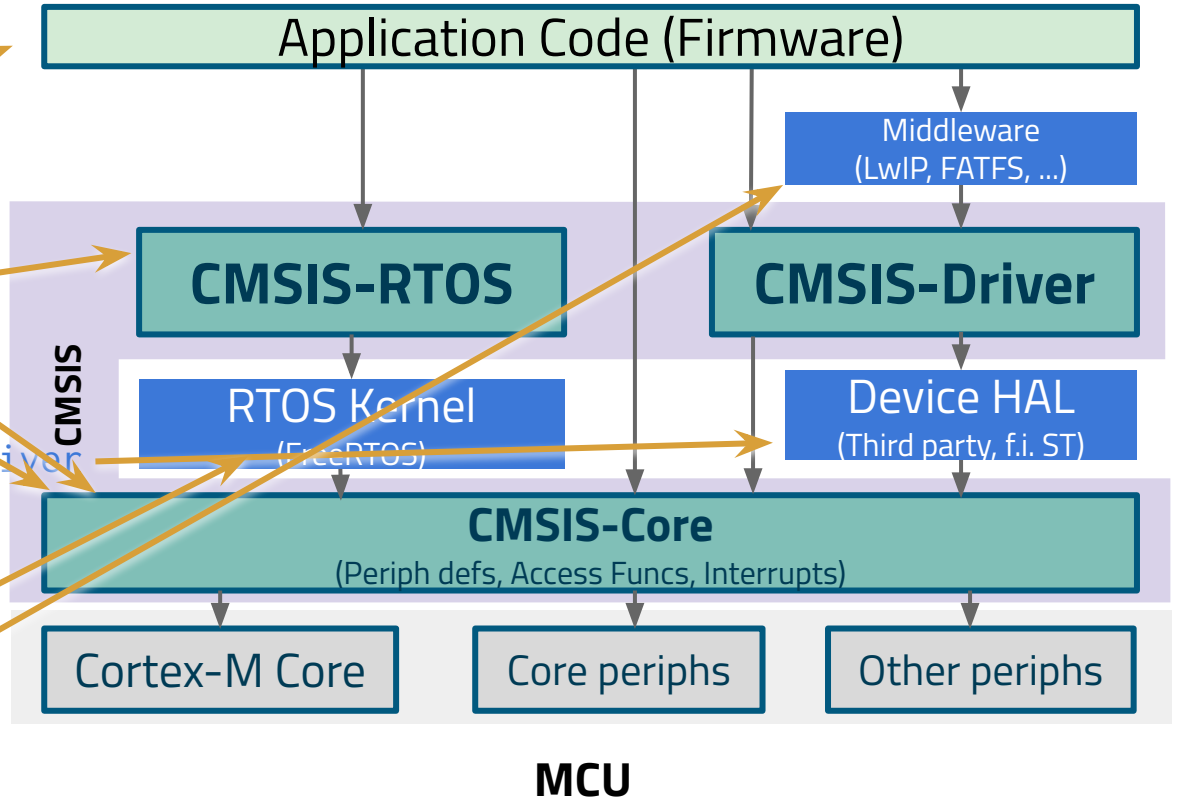
```
/* USER CODE BEGIN [NAME] */
printf("your code is safe here!\n");
/* USER CODE END [NAME] */
```

RECALL: everything outside the "USER CODE" sections will be overwritten by CubeMX

Project Structure

Project Root

- Inc
- Src
- Drivers
  - CMSIS
    - RTOS
    - Include
    - Device
  - STM32XXX_HAL_Driver
- Middlewares
  - Third_Party
    - FreeRTOS
    - ...



main.c

```
#include “main.h”
int main(void) {
    /* Reset all periph, Init the Flash interface and the Systick. */
    HAL_Init();
    /* Configure the system clock */
    SystemClock_Config();
    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    while (1)  { /* While 1 where the firmware lives */
    }
}
```

main.h

```
#ifndef __MAIN_H
#define __MAIN_H
#include "stm32f4xx_hal.h"
void Error_Handler(void);
/* Private defines */
#define BUTTON1_Pin GPIO_PIN_1
#define BUTTON1_GPIO_Port GPIOC
#define EXT_LED_Pin GPIO_PIN_2
#define EXT_LED_GPIO_Port GPIOC
#endif /* __MAIN_H */
```

stm32f4xx_hal_conf.h

- File auto generated by the IDE based on the configured peripherals during the set-up phase
- Defines which modules (peripherals) are used (f.i. `#define HAL_RCC_MODULE_ENABLED`)
- Includes all the header files that are needed (conditional include with `#ifdef`'s)
- Defines all the important constants for system configuration (clocks frequencies, start-up timeout, VDD value, Core configurations)

Some useful HAL functions for GPIO

All HAL functions on GPIOs take as input a pointer to the structure `GPIO_TypeDef` (stm32f446xx.h) comprising all GPIO port register

```
typedef struct {
  __IO uint32_t MODER;  /* GPIO port mode register,offset: 0x00 */
  __IO uint32_t OTYPER; /* GPIO port output type register, offset:0x04*/
  __IO uint32_t OSPEEDR;/* GPIO port output speed register, offset:0x08*/
  __IO uint32_t PUPDR; /* GPIO port pull-up/pull-down, offset: 0x0C */
  __IO uint32_t IDR;  /* GPIO port input data register, offset: 0x10 */
  __IO uint32_t ODR;  /* GPIO port output data register, offset: 0x14 */
  __IO uint32_t BSRR; /*  GPIO port bit set/reset register, offset: 0x18 */
  __IO uint32_t LCKR; /* GPIO port configuration lock register, offset: 0x1C */
  __IO uint32_t AFR[2]; /* GPIO alternate function registers, offset: 0x20-0x24 */
} GPIO_TypeDef;
```

Some useful HAL functions for GPIO

```
void HAL_GPIO_WritePin( GPIO_TypeDef* GPIOx,
                        uint16_t GPIO_Pin,
                        GPIO_PinState PinState)
```

- Expected result: Writes a value to a given pin of a port
- `GPIO_TypeDef* GPIOx`: the pointer to the GPIO port registers
- `uint16_t GPIO_Pin`: pin number.
- `GPIO_PinState PinState`: enumerated variable which can be either `GPIO_PIN_RESET` (logic low) or `GPIO_PIN_SET` (logic high)

```
typedef enum
{
  GPIO_PIN_RESET = 0,
  GPIO_PIN_SET
}GPIO_PinState;
```

Copyright notice

Copyright © by **Luca De Cicco**, 2019,2020

All rights reserved. No part of the material covered in this slides may be reproduced, stored in a retrieval system, or transmitted, recorder, or otherwise, without prior written permission of the author.